



Testing, Debugging y Documentación con IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Depuración de errores y detección de vulnerabilidades.



- I. ¿Por qué usar IA para depuración?
- II. Tipos de errores que la IA detecta bien
- III. Cómo pedir a la IA que depure un error
- IV. Ejemplo real de depuración con IA
- V. IA como analista de causa raíz (Root Cause Analysis)
- VI. Detección de vulnerabilidades con IA
- VII. Cómo pedir a la IA una auditoría de seguridad
- VIII. Buenas prácticas al usar IA para depuración y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Idea clave

La IA funciona como un **asistente técnico experto**, capaz de analizar errores con precisión, explicar la causa raíz de los fallos y señalar vulnerabilidades que suelen pasar inadvertidas tanto para estudiantes, alumnos o como para desarrolladores con experiencia.

Su capacidad para interpretar mensajes complejos, revisar patrones de riesgo y detectar comportamientos anómalos convierte la depuración y la seguridad en procesos más rápidos, claros y accesibles..

■ 2.1 ¿Por qué usar IA para depuración?

La depuración es una de las tareas más costosas y frustrantes para los programadores, pues como funciona el código no lo depuramos, es un gran error.

La IA convierte este proceso en algo más rápido, comprensible y guiado, actuando como un asistente experto que ayuda a interpretar errores y encontrar soluciones.

- ◆ Analiza errores rápidamente
- ◆ Explica trazas complejas
- ◆ Sugiere soluciones paso a paso
- ◆ Identifica malas prácticas y razona sobre errores
- ◆ Reduce tiempo de debugging

2.1 ¿Por qué usar IA para depuración?

◆ Analiza errores rápidamente

La IA puede leer mensajes de error extensos y confusos, interpretarlos y resumirlos en un lenguaje claro.

Esto reduce el tiempo que el programador pasa "atascado" sin saber por dónde empezar.

```
def calcular_promedio(numeros):  
    # Convertir todos a números por si acaso  
    numeros = [float(n) if isinstance(n, str) else n for n in numeros]  
    return sum(numeros) / len(numeros)
```

Ahora funciona con ambos formatos

```
resultado1 = calcular_promedio([10, 20, 30, 40])  
resultado2 = calcular_promedio([10, 20, "30", 40]) # También funciona
```

🔍 PROBLEMA: Estás mezclando números y texto en una lista.

📍 DONDE:

- Línea 7: Cuando llamas a `calcular\_promedio()`
- Línea 3: Al intentar sumar los elementos

✗ ERROR CONCRETO:

No se puede sumar `10 + 20 + "treinta" + 40` porque "treinta" es texto.

✅ SOLUCIÓN:

Cambia la lista a solo números:

`[10, 20, 30, 40]` en lugar de `[10, 20, "treinta", 40]`

💡 CONSEJO ADICIONAL:

Si necesitas manejar datos mixtos, puedes convertir a números primero.

■ 2.1 ¿Por qué usar IA para depuración?

◆ Explica trazas complejas

Las trazas de error suelen ser difíciles de entender para principiantes.

La IA puede:

- Identificar el origen real del fallo
- Explicar qué significa cada parte de la traza
- Señalar el módulo o función responsable
- Esto acelera la comprensión del problema.

```
def saludar(nombre):  
    mensaje = "Hola " + nombre  
    print(mensaje)  
  
def main():  
    nombre = 123 # ¡Error! Es un número, no texto  
    saludar(nombre)
```

```
main()
```

2.1 ¿Por qué usar IA para depuración?

```
Traceback (most recent call last):
  File "programa.py", line 8, in <module>
    main()
  File "programa.py", line 6, in main
    saludar(nombre)
  File "programa.py", line 2, in saludar
    mensaje = "Hola " + nombre
TypeError: can only concatenate str (not "int") to str
```

Problema: Estás juntando texto con un número.

Solución: Convierte el número a texto con str().

`nombre = "123" # Con comillas es texto`

```
def saludar(nombre):
    mensaje = "Hola " + str(nombre) # Convierte a texto
    print(mensaje)
```

- "Hola " es como una palabra escrita
- 123 es como un número escrito
- **No puedes** pegar una palabra con un número escrito
- **Sí puedes** pegar palabra con palabra

Tu error fue como intentar:

"Pegar" +  (123) ❌

"Pegar" + "123" ✅

```
def saludar(nombre):
    mensaje = "Hola " + nombre
    print(mensaje)

def main():
    nombre = 123 # ¡Error! Es un número, no texto
    saludar(nombre)

main()
```

■ 2.1 ¿Por qué usar IA para depuración?

◆ Sugiere soluciones paso a paso

La IA no solo detecta el error: propone cómo solucionarlo, indicando:

- qué línea corregir (ZeroDivisionError: division by zero)
- qué validación falta
- qué dependencia está fallando
- qué alternativa es más segura o eficiente

Es como tener un mentor técnico disponible en todo momento.

🔍 Línea problemática: Línea 3 - `total / len(notas)`
Problema: `len(notas)` puede ser 0 si la lista está vacía

✗ VALIDACIÓN QUE FALTA:

Necesitas verificar que la lista no esté vacía antes de dividir

```
def calcular_promedio(notas):  
    if len(notas) == 0: # ← Validación añadida  
        return 0 # 0 algún valor por defecto  
  
    total = sum(notas)  
    promedio = total / len(notas)  
    return promedio
```

■ 2.1 ¿Por qué usar IA para depuración?

◆ Identifica malas prácticas

La IA puede detectar patrones peligrosos o poco profesionales, como:

- variables sin inicializar
- validaciones insuficientes
- consultas inseguras
- manejo incorrecto de errores
- duplicación de lógica

Esto ayuda a los alumnos a mejorar su criterio técnico.

```
def calcular_descuento(total_compra, cliente_premium):  
    if cliente_premium:  
        descuento = total_compra * 0.20  
  
    print(f"Descuento aplicado: ${descuento}")  
    return total_compra - descuento
```

2.1 ¿Por qué usar IA para depuración?

! MALA PRÁCTICA DETECTADA: Variable sin inicializar

📌 PROBLEMA:

- Línea 6: Variable 'descuento' puede no existir si cliente_premium es False
- Esto causará: NameError: name 'descuento' is not defined

🔍 PATRÓN PELIGROSO:

- Variables definidas solo en ciertas condiciones
- Uso sin verificar existencia previa

✓ SOLUCIÓN:

Inicializa todas las variables al principio:

```
def calcular_descuento(total_compra, cliente_premium):  
    descuento = 0 # ← Valor por defecto  
    if cliente_premium:  
        descuento = total_compra * 0.20  
    print(f"Descuento aplicado: ${descuento}")  
    return total_compra - descuento
```

💡 MEJOR PRÁCTICA:

Siempre inicializa variables con valores por defecto

Usa 'if-else' explícito en lugar de 'if' parcial

■ 2.1 ¿Por qué usar IA para depuración?

◆ Reduce tiempo de debugging

Al interpretar errores y proponer soluciones, la IA reduce drásticamente el tiempo invertido en depurar. Se avanza más rápido y dedica más tiempo a comprender el código que a pelearse con él.

La IA no solo corrige: **explica por qué ocurre el fallo**. Esto permite que :

- Entienda la causa raíz
- Aprenda a evitar el error en el futuro
- Desarrolle pensamiento crítico
- Mejore su autonomía como programador

2.1 Depuración de errores y detección de vulnerabilidades

Propuesta:

Depuración tradicional	Depuración con IA
Buscar en Stack Overflow	Analiza tu error específico
Leer documentación extensa	Resume el problema en lenguaje claro
Probar soluciones al azar	Sugiere solución paso a paso
Horas de frustración	Minutos de comprensión
Depende de experiencia previa	Accesible para todos los niveles
Difícil interpretar errores complejos	Explicaciones claras y adaptadas
Proceso lento y manual	Respuestas rápidas y automatizadas
Fragmentos de información dispersa	Información centralizada y contextual
Prueba y error constante	Diagnóstico más preciso
Poco feedback inmediato	Retroalimentación en tiempo real
Requiere cambiar entre múltiples pestañas	Todo en una sola conversación
Curva de aprendizaje más pronunciada	Aprendizaje guiado y progresivo
Soluciones genéricas	Soluciones personalizadas
Difícil mantener el foco	Flujo de trabajo más fluido

■ 2.2 Tipos de errores que la IA detecta bien

- ✓ Errores lógicos.
- ✓ Errores de ejecución.
- ✓ Errores de asincronía
- ✓ Errores de diseño

■ 2.2 Tipos de errores que la IA detecta bien

✓ Errores lógicos

Condiciones incorrectas

Bucles mal definidos

Off-by-one

✓ Errores de ejecución

Null / undefined

Tipos incorrectos

Excepciones no controladas

```
edad = 15

if edad >= 18:
    print("Mayor de edad")
elif edad < 18: # ← ¡REDUNDANTE!
    print("Menor de edad")
elif edad == 18: # ← ¡NUNCA SE EJECUTA!
    print("Justo 18")
```

```
let usuario = {
  nombre: "Ana"
};

console.log(usuario.edad.meses); // ← ¡CRASH!
// TypeError: Cannot read property 'meses' of undefined
```

2.2 Tipos de errores que la IA detecta bien

✓ Errores de asincronía

Promesas sin await

Callbacks mal encadenados

Race conditions simples

✓ Errores de diseño

Funciones demasiado grandes

Dependencias circulares

Falta de modularidad

```
async function obtenerNombre() {  
    return "Juan";  
}  
  
// ¡ERROR!  
let nombre = obtenerNombre();  
console.log(nombre); // → "[object Promise]" 🤯
```

```
# ¡CÓDIGO REPETIDO!  
def saludar_usuario(nombre):  
    print(f"¡Hola {nombre}!")  
    print("Bienvenido al sistema")  
    # ... 10 líneas más ...  
  
def despedir_usuario(nombre):  
    print(f"¡Adiós {nombre}!")  
    print("Vuelve pronto")  
    # ... LAS MISMAS 10 líneas ... 🤯
```

Depuración de errores y detección de vulnerabilidades



```
# ✗ ERROR LÓGICO: División por cero
x = 10
y = 0
resultado = x / y # ✗ ZeroDivisionError
```

```
# ✗ ERROR DE TIPO
numero = "5"
resultado = numero + 10 # ✗ TypeError
```

```
# ✗ ERROR DE ÍNDICE
lista = [1, 2, 3]
print(lista[5]) # ✗ IndexError
```

```
# ✗ VARIABLE NO DEFINIDA
print(total) # ✗ NameError
```

```
# ✗ ERROR DE SINTAXIS
if True
    print("Hola") # ✗ SyntaxError
```

```
# ✗ ERROR DE MUTABILIDAD
tupla = (1, 2, 3)
tupla[0] = 10 # ✗ TypeError
```

```
# ✗ ERROR DE IMPORTACIÓN
import matematicas # ✗ ModuleNotFoundError
```

```
# ✗ ERROR EN FUNCIÓN
def suma(a, b):
    return a + b
```

```
suma(5) # ✗ TypeError: falta argumento
```

```
# ✗ ERROR DE ASIGNACIÓN
x + 5 = 10 # ✗ SyntaxError
```

```
# ✗ ERROR DE ASINCRONÍA (JavaScript)
async function obtenerDatos() {
    const res = fetch('/api'); // ✗ Falta await
    console.log(res.json()); // ✗ Promesa no resuelta
}
```

■ 2.3 Cómo pedir a la IA que depure un error

Para que la IA pueda analizar un error de forma precisa, el prompt debe ser **completo, específico y contextualizado**. Cuanta más información reciba, más útil será su análisis y sus propuestas de solución.

■ 2.3 Cómo pedir a la IA que depure un error

◆ 1. El código completo o el fragmento relevante

La IA necesita ver **exactamente** qué parte del código está fallando.

- función o método
- clase o módulo
- bloque donde ocurre el error
- dependencias relacionadas

Evitar:

"Tengo un error en mi función,
¿qué puede ser?"

Mejor:

"Aquí tienes la función completa donde
ocurre el error."

 **ANALOGÍA:**

 **Como decir al médico:**

"Me duele"

→ ¿Dónde? ¿Cuándo? ¿Cómo?

 **Como decir al médico:**

"Tengo dolor en la rodilla derecha cuando subo escaleras,
empezó ayer"

→ ¡Diagnóstico preciso!

■ 2.3 Cómo pedir a la IA que depure un error

◆ 2. El error exacto (mensaje y traza)

La IA debe recibir el mensaje tal cual aparece:

- mensaje de error
- traza completa
- línea afectada
- módulo o archivo implicado

Esto permite identificar la causa raíz con precisión.

ANALOGÍA:

 Como decir al mecánico:

"El auto no funciona"

→ ¿Qué parte? ¿Qué ruido hace? ¿Cuándo pasa?

 Como decir al mecánico:

"El auto hace 'clack-clack' al frenar, y la luz del ABS está encendida"

→ ¡Diagnóstico inmediato!

■ 2.3 Cómo pedir a la IA que depure un error

◆ 3. El contexto (qué debería hacer el código)

La IA necesita saber qué esperabas que ocurriera.

- comportamiento esperado
- datos de entrada
- flujo previsto

resultado correcto

Sin contexto, la IA solo puede adivinar.

ANALOGÍA DEL GPS

SIN CONTEXTO (Como usar GPS mal):

"Llevo 10 minutos manejando y no llego"

→ **GPS:** ¿A dónde quieres ir? ¿Desde dónde partes? ¿En auto o a pie?

CON CONTEXTO (Como usar GPS bien):

"Quiero ir del **Centro Comercial Galerías** al **Aeropuerto Internacional** en auto, evitando cuotas**"

→ **GPS:** Toma la Av. Principal, luego la carretera 45-D...

■ 2.3 Cómo pedir a la IA que depure un error

◆ 4. El entorno (lenguaje, framework, versión)

Muchos errores dependen del entorno:

- Node.js, Python, Java, etc.
- framework (Express, Django, Spring...)
- versión de librerías
- base de datos o servicios externos

Esto evita sugerencias incompatibles.

ANALOGÍA CULINARIA

RECETA SIN CONTEXTO:

"¿Cómo hago pan?"

→ **Chef:** ¿Pan francés? ¿Integral? ¿Sin gluten? ¿Con qué horno?

RECETA CON CONTEXTO:

"Quiero hacer pan sin gluten en horno convencional"

→ **Chef:** Usa harina de arroz, huevo y hornea a 180°C

■ 2.3 Cómo pedir a la IA que depure un error

◆ 5. Qué tipo de ayuda necesitas

- explicar el error
- encontrar la causa raíz
- proponer una solución
- sugerir alternativas más seguras
- refactorizar el código
- detectar vulnerabilidades

Indicarlo mejora
la calidad de la
respuesta.

ANALOGÍA DEL PROFESOR

❌ ALUMNO VAGO:

"No entiendo matemáticas"

→ **Profesor:** ¿Álgebra? ¿Cálculo? ¿Quieres teoría? ¿Ejercicios?
¿Explicación paso a paso?

✅ ALUMNO ESPECÍFICO:

"Explicame la regla de la cadena" (explicación)

"¿Este ejercicio está bien resuelto?" (revisión)

"Dame otro ejercicio similar" (práctica)

■ 2.3 Cómo pedir a la IA que depure un error

✚ Ejemplo de prompt

“Analiza este error, explica la causa raíz y propón una solución segura siguiendo buenas prácticas.

Aquí tienes el código, la traza completa y el comportamiento esperado.”

Este prompt funciona porque incluye:

- código
- error
- contexto
- entorno
- objetivo

Depuración de errores
y detección de
vulnerabilidades

■ 2.3 Cómo pedir a la IA que depure un error

EJEMPLO DE PROMPT COMPLETO

"Analiza este error. Tengo una función en Python que debería calcular el promedio de una lista de notas, pero recibo un error al ejecutarla con una lista vacía.

CÓDIGO:

```
def calcular_promedio(notas):  
    return sum(notas) / len(notas)
```

ERROR:

```
ZeroDivisionError: division by zero  
File "main.py", line 2, in calcular_promedio  
    return sum(notas) / len(notas)
```

COMPORTAMIENTO ESPERADO:

Si la lista está vacía, debería devolver 0 o mostrar un mensaje, no romper.

ENTORNO: Python 3.10

AYUDA: Explica la causa raíz y propón una solución siguiendo buenas prácticas."

■ 2.4 Ejemplo real de depuración con IA

```
async function getUser(id) {  
  return db.users.find(u => u.id === id)  
}
```

Error recibido

```
TypeError: Cannot read properties of undefined (reading 'find')
```

Qué debería detectar la IA

- ◆ 1. **db.users** no existe o no está inicializado

La IA identifica que el error ocurre porque **db.users** es undefined.

Esto puede deberse a:

- ✓ la base de datos no está cargada
- ✓ la propiedad **users** no existe
- ✓ el mock está mal definido
- ✓ el módulo **db** no se importó correctamente

Este es el origen directo del error.

■ 2.4 Ejemplo real de depuración con IA

◆ 2. Falta de **await** en una operación asíncrona

Aunque la función es **async**, el acceso a **db.users** parece síncrono.

La IA debería señalar:

- ✓ si **db.users** proviene de una consulta asíncrona
- ✓ si falta un **await** antes de acceder a los datos
- ✓ si la función debería manejar promesas correctamente

Esto puede generar errores silenciosos o datos incompletos.

◆ 3. Falta de validación del parámetro **id**

La IA deberá advertir de que no se valida:

- ✓ si **id** existe
- ✓ si es del tipo correcto
- ✓ si es un valor permitido

Esto puede provocar:

- ✓ búsquedas incorrectas
- ✓ errores inesperados
- ✓ vulnerabilidades (por ejemplo, IDs manipulados)

■ 2.4 Ejemplo real de depuración con IA

◆ 4. Falta de manejo de errores

- ✓ La función no contempla:
- ✓ qué ocurre si el usuario no existe
- ✓ qué pasa si la base de datos falla
- ✓ cómo responder ante excepciones

La IA debería sugerir:

- ✓ try/catch
- ✓ validaciones previas
- ✓ mensajes de error claros
- ✓ retorno seguro en caso de fallo

Qué vemos con este ejemplo

La IA no solo identifica el error superficial, sino que analiza:

- ☐ el origen real del fallo
- ☐ problemas de diseño
- ☐ riesgos de seguridad
- ☐ malas prácticas
- ☐ mejoras necesarias

Es un ejemplo perfecto para mostrar cómo la IA ayuda a **razonar**, no solo a corregir.

Depuración de errores
y detección de
vulnerabilidades

■ 2.4 Ejemplo real de depuración con IA

```
// CÓDIGO QUE PROVOCA EL ERROR (sin documentación)
const db = {}; // ✗ db.users no existe

async function findUserById(id) {
  // ✗ Error: db.users es undefined
  return db.users.find(user => user.id === id);
}

app.get('/api/users/:id', async (req, res) => {
  const user = await findUserById(req.params.id);
  res.json(user);
});
```

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

◆ Qué puede explicar la IA durante un análisis de causa raíz

✓ 1. Qué provoca el error

La IA identifica la causa inmediata del fallo:

- ✓ variable no definida
- ✓ acceso a propiedad inexistente
- ✓ tipo incorrecto
- ✓ dependencia no inicializada
- ✓ error de lógica

Esto ayuda al programador a entender el "qué".

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

- ◆ Qué puede explicar la IA durante un análisis de causa raíz
- ✓ 2. Qué línea o módulo lo origina

La IA puede señalar con precisión:

- ✓ la línea exacta donde se produce el error
- ✓ el módulo o archivo implicado
- ✓ la función que desencadena el fallo

Esto reduce el tiempo de búsqueda y evita confusiones.

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

- ◆ Qué puede explicar la IA durante un análisis de causa raíz
- ✓ 3. Qué condiciones lo reproducen

La IA puede describir el escenario que activa el error:

- ✓ entradas específicas
- ✓ estados no contemplados
- ✓ valores límite
- ✓ datos corruptos
- ✓ dependencias fallidas

Esto permite reproducir el fallo de forma controlada.

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

◆ Qué puede explicar la IA durante un análisis de causa raíz

✓ 4. Qué alternativas existen

La IA puede proponer soluciones basadas en buenas prácticas:

- ✓ validaciones adicionales
- ✓ manejo de errores
- ✓ refactorización
- ✓ uso de patrones más seguros
- ✓ cambios en la arquitectura

El alumno aprende no solo a corregir, sino a mejorar el diseño.

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

◆ Qué puede explicar la IA durante un análisis de causa raíz

✓ 5. Cómo evitarlo en el futuro

La IA puede sugerir medidas preventivas:

- ✓ tests unitarios específicos
- ✓ validaciones de entrada
- ✓ manejo de excepciones
- ✓ modularización
- ✓ documentación clara

Esto convierte el error en una oportunidad de aprendizaje.

■ 2.5 IA como analista de causa raíz (Root Cause Analysis)

◆ Qué puede explicar la IA durante un análisis de causa raíz

✚ Prompt recomendado:

“Haz un análisis de causa raíz de este error y sugiere una solución robusta siguiendo buenas prácticas.”

Este prompt funciona porque pide:

- ✓ análisis profundo
- ✓ explicación clara
- ✓ solución sólida
- ✓ enfoque profesional

🎯 Mensaje clave

La IA no solo corrige errores: enseña a pensar como un analista profesional. Comprender la causa raíz es más importante que aplicar un parche rápido.

```

productos = {
    "manzana": {"stock": 50, "precio": 1.5},
    "banana": {"stock": 30, "precio": 0.8},
    "naranja": {"stock": 40} # ¡Falta precio!
}

def vender_producto(nombre_producto, cantidad):
    # Verificar si producto existe
    if nombre_producto not in productos:
        print("Producto no encontrado")
        return

    # Verificar stock disponible
    if productos[nombre_producto]["stock"] < cantidad:
        print("Stock insuficiente")
        return

    # Calcular total
    total = productos[nombre_producto]["precio"] * cantidad # ←

    # Actualizar stock
    productos[nombre_producto]["stock"] -= cantidad

    print(f"Venta realizada: {cantidad} x {nombre_producto}")
    print(f"Total: ${total:.2f}")
    return total

```

```

# Pruebas
print("--- Prueba 1: Venta normal ---")
vender_producto("manzana", 2) # ✅ Funciona

print("\n--- Prueba 2: Venta sin stock ---")
vender_producto("banana", 100) # ✅ Stock insuficiente

print("\n--- Prueba 3: Error crítico ---")
vender_producto("naranja", 3) # ❌ ¡ERROR AQUÍ!

print("\n--- Prueba 4: Nunca se ejecuta ---")
vender_producto("manzana", 1) # No se alcanza por el crash

```

```

--- Prueba 1: Venta normal ---
Venta realizada: 2 x manzana
Total: $3.00

```

```

--- Prueba 2: Venta sin stock ---
Stock insuficiente

```

```

--- Prueba 3: Error crítico ---
Traceback (most recent call last):
  File "sistema_inventario.py", line 30, in <module>
    vender_producto("naranja", 3)
  File "sistema_inventario.py", line 15, in vender_producto
    total = productos[nombre_producto]["precio"] * cantidad
KeyError: 'precio'

```

🔍 CAUSA INMEDIATA: `KeyError: 'precio'`
📌 PROBLEMA: Intentas acceder a `productos["naranja"]["precio"]`, pero "naranja" no tiene clave "precio"

📌 ESTRUCTURA ACTUAL:
`"naranja": {"stock": 40}` # ← Solo tiene "stock", falta "precio"

📌 LÍNEA EXACTA: Línea 15 en `vender_producto()`
`total = productos[nombre_producto]["precio"] * cantidad`

📁 MÓDULO: `sistema_inventario.py`

🔗 FUNCIÓN: `vender_producto(nombre_producto, cantidad)`

📄 RESPONSABLE: La función asume que TODOS los productos tienen "precio"

🎯 ESCENARIO QUE ACTIVA EL ERROR:

1. Producto existe en el diccionario ✓
2. Tiene suficiente stock ✓
3. Pero NO tiene clave "precio" definida ✗

📋 DATOS QUE LO CAUSAN:
`{"stock": 40}` # Diccionario incompleto

🔍 CÓMO REPRODUCIRLO:
`vender_producto("naranja", cualquier_cantidad)`
Donde "naranja" = `{"stock": 40}` (sin "precio")

✓ SOLUCIONES POSIBLES:

Opción A: Validación explícita (Defensiva)

```
def vender_producto(nombre_producto, cantidad):  
    if nombre_producto not in productos:  
        return "Producto no encontrado"  
  
    producto = productos[nombre_producto]  
  
    # Validar estructura completa  
    if "stock" not in producto or "precio" not in producto:  
        return "Datos del producto incompletos"  
  
    if producto["stock"] < cantidad:  
        return "Stock insuficiente"  
  
    total = producto["precio"] * cantidad  
    producto["stock"] -= cantidad  
    return total
```

Opción C: Valores por defecto

```
productos = {  
    "naranja": {"stock": 40, "precio": 1.2}, # ← Ahora sí tiene precio  
}
```

Opción B: Clase Producto (Orientación a Objetos)

```
class Producto:  
    def __init__(self, nombre, stock, precio):  
        if precio <= 0:  
            raise ValueError("Precio debe ser positivo")  
        self.nombre = nombre  
        self.stock = stock  
        self.precio = precio  
  
    def vender(self, cantidad):  
        if self.stock < cantidad:  
            raise ValueError("Stock insuficiente")  
        self.stock -= cantidad  
        return self.precio * cantidad
```



MEDIDAS PREVENTIVAS:

1. VALIDACIÓN DE ESTRUCTURA:
2. TESTS UNITARIOS:
3. DOCUMENTACIÓN CLARA:
4. MEJOR DISEÑO:

CÓDIGO PROBLEMÁTICO

```
productos = {  
    "manzana": {"stock": 10, "precio": 1.5},  
    "naranja": {"stock": 40}, # ✗ Falta "precio"  
}
```

```
def vender_producto(nombre, cantidad):  
    total = productos[nombre]["precio"] * cantidad # ✗ KeyError: 'precio'  
    productos[nombre]["stock"] -= cantidad  
    return total
```

```
vender_producto("naranja", 2) # ✗ EXPLOTA
```

🔍 ANÁLISIS PASO A PASO QUE LA IA DEBERÍA HACER:

1. CAUSA INMEDIATA: KeyError: 'precio'

2. LÍNEA EXACTA: total = productos[nombre]["precio"]

3. CONDICIÓN: "naranja" existe pero no tiene clave "precio"

4. SOLUCIÓN: Validar estructura antes de acceder

■ 2.6 Detección de vulnerabilidades con IA

La IA puede actuar como un **analista de seguridad automatizado**, revisando código, endpoints, flujos y configuraciones para identificar riesgos que suelen pasar desapercibidos.

Esto no permite pensar como pentesters y a reforzar la seguridad desde etapas tempranas del desarrollo.

Pentesters (o *penetration testers*) son profesionales de ciberseguridad que simulan ataques controlados contra sistemas, redes o aplicaciones para identificar vulnerabilidades antes de que lo hagan atacantes reales.

■ 2.6 Detección de vulnerabilidades con IA

◆ Inyecciones SQL

La IA puede detectar patrones peligrosos como:

- ✓ concatenación directa de strings en consultas
- ✓ parámetros no sanitizados
- ✓ uso incorrecto de ORMs

consultas construidas dinámicamente sin validación

Ejemplo:

```
db.query("SELECT * FROM users WHERE id = " + id)
```

■ 2.6 Detección de vulnerabilidades con IA

◆ XSS (Cross-Site Scripting)

La IA identifica:

- ✓ contenido no escapado en plantillas
- ✓ inputs que se renderizan sin limpiar o neutraliza
- ✓ uso inseguro de `innerHTML`
- ✓ falta de validación en formularios

Esto ayuda a prevenir ataques que comprometen al usuario final.

■ 2.6 Detección de vulnerabilidades con IA

◆ CSRF (Cross-Site Request Forgery)

La IA puede señalar:

- ✓ formularios sin tokens CSRF
- ✓ endpoints sensibles sin protección
- ✓ sesiones vulnerables
- ✓ métodos inseguros en operaciones críticas

Ideal para enseñar seguridad en aplicaciones web.

.

■ 2.6 Detección de vulnerabilidades con IA

◆ Exposición de datos sensibles

La IA puede advertir sobre:

- ✓ envío de contraseñas en claro
- ✓ respuestas que incluyen `passwordHash`, `tokens`, `API keys`
- ✓ logs con información sensible
- ✓ endpoints demasiado verbosos

Ayuda a reforzar la privacidad y el cumplimiento normativo.

■ 2.6 Detección de vulnerabilidades con IA

◆ Dependencias inseguras

La IA puede revisar:

- ✓ librerías obsoletas
- ✓ versiones con CVEs conocidos
- ✓ paquetes sin mantenimiento
- ✓ dependencias que introducen riesgos indirectos

Esto fomenta buenas prácticas de mantenimiento.

■ 2.6 Detección de vulnerabilidades con IA

◆ Errores de autenticación y autorización

La IA puede detectar:

- ✓ endpoints sin protección
- ✓ roles mal definidos
- ✓ tokens no verificados
- ✓ sesiones inseguras
- ✓ lógica de permisos incorrecta

Esto es clave para enseñar seguridad basada en roles y flujos protegidos.

1. Inyecciones SQL



```
# ❌ PELIGROSO: Concatenación directa
id_usuario = input("ID: ") # Usuario podría poner: 1 OR 1=1
query = f"SELECT * FROM usuarios WHERE id = {id_usuario}"
cursor.execute(query) # ¡SQL Injection!

# ✅ SEGURO: Parámetros preparados
id_usuario = input("ID: ")
query = "SELECT * FROM usuarios WHERE id = ?"
cursor.execute(query, (id_usuario,)) # SQL safe
```

2. XSS (Cross-Site Scripting)

```
// ❌ PELIGROSO: innerHTML sin sanitizar
let comentario = document.getElementById('comentario').value;
// Usuario podría escribir: <script>robarCookies()</script>
document.body.innerHTML += comentario; // ¡Ejecuta el script!

// ✅ SEGURO: textContent o sanitizar
let comentarioSeguro = sanitize(comentario); // Primero sanitizar
document.body.textContent += comentarioSeguro;
```

3. CSRF (Cross-Site Request Forgery)

```
<!-- ❌ PELIGROSO: Sin token CSRF -->
<form action="/transferir-dinero" method="POST">
  <input type="text" name="monto">
  <input type="text" name="cuenta_destino">
  <button>Transferir</button>
</form>

<!-- ✅ SEGURO: Con token oculto -->
<form action="/transferir-dinero" method="POST">
  <input type="hidden" name="csrf_token" value="ABC123">
  <input type="text" name="monto">
  <input type="text" name="cuenta_destino">
  <button>Transferir</button>
</form>
```

4. Validaciones Insuficientes



```
# ❌ PELIGROSO: Sin validaciones
def cambiar_edad(usuario_id, nueva_edad):
    # Cualquiera podría poner: edad = -100 o edad = "abc"
    db.execute(f"UPDATE usuarios SET edad = {nueva_edad} WHERE id = {usuario_id}")

# ✅ SEGURO: Con validaciones
def cambiar_edad(usuario_id, nueva_edad):
    if not isinstance(nueva_edad, int):
        raise ValueError("Edad debe ser número")
    if nueva_edad < 0 or nueva_edad > 120:
        raise ValueError("Edad inválida")

    db.execute("UPDATE usuarios SET edad = ? WHERE id = ?",
              (nueva_edad, usuario_id))
```

5. Exposición de Datos Sensibles

```
// ❌ PELIGROSO: Enviar contraseña en claro
fetch('/login', {
  method: 'POST',
  body: JSON.stringify({
    email: 'usuario@email.com',
    password: 'miclave123' // ¡En texto plano!
  })
})

// ✅ SEGURO: Hash en cliente o HTTPS
fetch('/login', {
  method: 'POST',
  body: JSON.stringify({
    email: 'usuario@email.com',
    passwordHash: sha256('miclave123' + 'salt') // Hash con salt
  })
})
```

6. Dependencias Inseguras



```
// ❌ PELIGROSO: Versiones viejas con vulnerabilidades
{
  "dependencies": {
    "express": "4.16.0", // Versión vieja con CVE-2021-xxxx
    "lodash": "4.17.15"  // Podría tener vulnerabilidades
  }
}

// ✅ SEGURO: Versiones actualizadas
{
  "dependencies": {
    "express": "^4.18.0", // Última estable
    "lodash": "^4.17.21"  // Sin vulnerabilidades conocidas
  }
}
```

7. Errores de Autenticación

```
# ❌ PELIGROSO: Verificación básica
def es_admin(usuario_id):
    # Cualquiera podría enviar usuario_id = 1 (admin)
    usuario = db.get_usuario(usuario_id)
    return usuario and usuario.es_admin

# ✅ SEGURO: Verificar sesión actual
def es_admin():
    usuario_actual = get_usuario_actual() # De la sesión
    return usuario_actual and usuario_actual.es_admin
```

Inyección SQL (SQL Injection)

🚫 CÓDIGO VULNERABLE

```
import sqlite3
```

```
def login_usuario(email, password):
```

```
    conn = sqlite3.connect("database.db")
```

```
    cursor = conn.cursor()
```

❌ PELIGRO: Concatenación directa de strings

```
query = f"SELECT * FROM usuarios WHERE email = '{email}' AND password = '{password}'"
```

```
cursor.execute(query)
```

```
usuario = cursor.fetchone()
```

```
conn.close()
```

```
return usuario
```

🚩 ATAQUE POSIBLE:

```
# email = "admin' --"
```

```
# password = "loquesea"
```

```
# La consulta se convierte en:
```

```
# SELECT * FROM usuarios WHERE email = 'admin' --' AND password = 'loquesea'
```

```
# ¡El atacante inicia sesión sin contraseña!
```

✅ CÓDIGO SEGURO (usando parámetros)

```
def login_usuario_seguro(email, password):
```

```
    conn = sqlite3.connect("database.db")
```

```
    cursor = conn.cursor()
```

✅ CORRECTO: Usar parámetros

```
query = "SELECT * FROM usuarios WHERE email = ? AND password = ?"
```

```
cursor.execute(query, (email, password))
```

```
usuario = cursor.fetchone()
```

```
conn.close()
```

```
return usuario
```

También con SQLAlchemy/ORM

```
# usuario = db.session.query(Usuario).filter(
```

```
#     Usuario.email == email,
```

```
#     Usuario.password == password
```

```
# ).first()
```

XSS (Cross-Site Scripting)

```
# 🚫 CÓDIGO VULNERABLE (FastAPI/Flask)

from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.get("/comentario", response_class=HTMLResponse)
def mostrar_comentario(comentario: str):
    # ✗ PELIGRO: El comentario puede contener código JavaScript malicioso
    html = f"""
    <html>
        <body>
            <h1>Comentario del usuario:</h1>
            <p>{comentario}</p>
        </body>
    </html>
    """
    return html

# 🕸 ATAQUE POSIBLE:
# URL: /comentario?comentario=<script>alert('Hackeado')</script>
# El navegador ejecuta el script sin que el usuario lo sepa
```

```
# ✅ CÓDIGO SEGURO (escapando el contenido)

import html

@app.get("/comentario", response_class=HTMLResponse)
def mostrar_comentario_seguro(comentario: str):
    # ✅ CORRECTO: Escapar caracteres peligrosos
    comentario_escaped = html.escape(comentario)

    html_content = f"""
    <html>
        <body>
            <h1>Comentario del usuario:</h1>
            <p>{comentario_escaped}</p>
        </body>
    </html>
    """
    return html_content
```

```
# En FastAPI, también puedes usar Jinja2 que escapa automáticamente:
# return templates.TemplateResponse("comentario.html", {"comentario": comentario})
```

Exposición de datos sensibles

```
# 🚫 CÓDIGO VULNERABLE
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Usuario(BaseModel):
```

```
    id: int
    nombre: str
    email: str
    password_hash: str # 🚫 Nunca debe exponerse
    api_key: str # 🚫 Información sensible
    tarjeta_credito: str # 🚫 Datos financieros
```

```
@app.get("/api/usuarios/{id}")
```

```
def obtener_usuario(id: int):
```

```
    usuario = {
        "id": 1,
        "nombre": "Ana García",
        "email": "ana@ejemplo.com",
        "password_hash": "$2b$12$xyz...", # 🚫 Expuesto
        "api_key": "sk_live_abc123", # 🚫 Expuesto
        "tarjeta_credito": "4532****1234" # 🚫 Expuesto
    }
    return usuario # 🚫 Devuelve datos sensibles
```

```
# ✅ CÓDIGO SEGURO
```

```
from pydantic import BaseModel
```

```
# ✅ Crear un modelo específico para respuestas (sin datos sensibles)
```

```
class UsuarioResponse(BaseModel):
```

```
    id: int
    nombre: str
    email: str
    # ✅ NO incluir password_hash, api_key, tarjeta_credito
```

```
@app.get("/api/usuarios/{id}", response_model=UsuarioResponse)
```

```
def obtener_usuario_seguro(id: int):
```

```
    usuario = {
        "id": 1,
        "nombre": "Ana García",
        "email": "ana@ejemplo.com"
        # ✅ Datos sensibles se quedan en el servidor
    }
    return usuario
```

```
# También es importante:
```

- # - No loguear contraseñas o tokens
- # - Usar HTTPS en producción
- # - No incluir secretos en código fuente (usar variables de entorno)

Errores de autenticación y autorización

```
# 🚫 CÓDIGO VULNERABLE (sin autenticación)

from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.delete("/api/usuarios/{id}")
def eliminar_usuario(id: int):
    # ❌ PELIGRO: Cualquier persona puede eliminar cualquier usuario
    # No hay verificación de identidad ni permisos
    usuarios_db = [u for u in usuarios_db if u["id"] != id]
    return {"mensaje": "Usuario eliminado"}
```

```
# ✅ CÓDIGO SEGURO (con autenticación y autorización)

from fastapi import FastAPI, HTTPException, Depends, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt

security = HTTPBearer()

# Función para obtener usuario autenticado
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    try:
        # Verificar token JWT
        payload = jwt.decode(token, "SECRET_KEY", algorithms=["HS256"])
        user_id = payload.get("id")
        if not user_id:
            raise HTTPException(status_code=401, detail="Token inválido")
        return user_id
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Token inválido o expirado")

# ✅ Solo usuarios autenticados pueden eliminar
@app.delete("/api/usuarios/{id}")
def eliminar_usuario_seguro(
    id: int,
    current_user_id: int = Depends(get_current_user) # Autenticación
):
    # ✅ Verificar autorización: solo puedes eliminar tu propio usuario
    if current_user_id != id:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="No tienes permiso para eliminar este usuario"
        )

    # Solo administradores deberían poder eliminar otros usuarios
    # if not es_administrador(current_user_id):
    #     raise HTTPException(status_code=403, detail="Solo administradores")
```

Vulnerabilidad	Ejemplo Malo	Ejemplo Bueno
SQL Injection	WHERE id = \${input}	WHERE id = ?
XSS	.innerHTML = input	.textContent = input
CSRF	Form sin token	Form con csrf_token
Validaciones	Sin verificar	if valor > 0:
Datos sensibles	password: "123"	passwordHash: "abc123"
Dependencias	v1.0.0 (2018)	^4.18.0 (2023)
Sanitización	Usar input directo	sanitize(input)
Autenticación	if id == 1:	Verificar sesión

■ 2.7 Cómo pedir a la IA una auditoría de seguridad



Una auditoría de seguridad con IA es el proceso mediante el cual un modelo de lenguaje avanzado analiza sistemáticamente código, configuraciones, arquitecturas y flujos de datos para identificar vulnerabilidades, malas prácticas y riesgos de seguridad. A diferencia de herramientas automatizadas tradicionales (como scanners de vulnerabilidades). Pero para que el análisis sea útil, el prompt debe estar bien construido, con contexto, alcance y criterios claros.

■ 2.7 Cómo pedir a la IA una auditoría de seguridad

◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 1. El código completo o el módulo relevante

La IA necesita ver el código tal cual está escrito:

- funciones
- controladores
- consultas a la base de datos
- validaciones
- middlewares
- Configuraciones

Cuanto más completo sea el fragmento, más precisa será la auditoría.

■ 2.7 Cómo pedir a la IA una auditoría de seguridad

◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 2. El contexto funcional del código

La IA debe saber qué se supone que hace el código:

- ¿Es un login?
- ¿Es un endpoint público o privado?
- ¿Procesa datos sensibles?
- ¿Interactúa con la base de datos?

Sin contexto, la IA puede detectar riesgos falsos o ignorar vulnerabilidades reales.

■ 2.7 Cómo pedir a la IA una auditoría de seguridad

- ◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 3. El entorno tecnológico

La seguridad depende del stack:

- lenguaje (JS, Python, Java...)
- framework (Express, Django, Spring...)
- ORM (Prisma, Sequelize, TypeORM...)
- base de datos (MongoDB, PostgreSQL...)

Esto permite que la IA aplique las reglas de seguridad adecuadas.

2.7 Cómo pedir a la IA una auditoría de seguridad

◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 4. El estándar o marco de referencia

Para auditorías profesionales, es clave indicar el estándar:

- OWASP Top 10(Vulnerabilidades web)
- CWE(catálogo técnico de debilidades de software)
- SANS(lista errores de programación)
- NIST(marco de ciberseguridad y estándares oficiales)

OWASP es el más común en formación y desarrollo web.



Frase clave

OWASP te dice qué falla,
CWE cómo se llama,
SANS cómo evitarlo
NIST cómo gestionarlo.

Depuración de errores y detección de vulnerabilidades



■ 2.7 Cómo pedir a la IA una auditoría de seguridad

◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 5. El tipo de análisis que necesitas

La IA puede realizar distintos niveles de auditoría:

- detección de vulnerabilidades
- explicación de riesgos
- propuestas de mitigación
- refactorización segura
- generación de tests de seguridad
- simulación de ataques (inputs maliciosos)

Indicarlo mejora la calidad del análisis.



■ 2.7 Cómo pedir a la IA una auditoría de seguridad

◆ ¿Qué debe incluir un prompt para auditoría de seguridad?

✓ 6. El nivel de detalle esperado

Puedes pedir:

- análisis superficial
- análisis profundo
- explicación pedagógica
- recomendaciones prácticas
- ejemplos de código seguro

Esto permite adaptar la auditoría al nivel del programador, para ejecutarlo a nivel empresarial.

2.7 Cómo pedir a la IA una auditoría de seguridad

Prompt recomendado

“Analiza este código y detecta vulnerabilidades siguiendo OWASP Top 10. Explica cada riesgo, indica la causa raíz, propone soluciones seguras y justifica por qué son necesarias.”

Este prompt es profesional porque:

- ✓ define el estándar (OWASP Top 10)
- ✓ pide explicación + causa raíz
- ✓ solicita soluciones seguras
- ✓ exige justificación técnica
- ✓ orienta a buenas prácticas

2.8 Buenas prácticas al usar IA para depuración y errores comunes

- ☐ Proporcionar contexto completo
 - ☐ Pedir explicación paso a paso
 - ☐ Validar siempre la solución
 - ☐ No aceptar cambios sin entenderlos
 - ☐ Pedir alternativas
 - ☐ Pedir mejoras de seguridad
 - ☐ Documentar el proceso
-
- ☐ Enviar solo el error sin el código
 - ☐ No explicar qué debería hacer el programa
 - ☐ Aceptar la solución sin revisarla
 - ☐ No probar casos Edge
 - ☐ No validar seguridad
 - ☐ No reproducir el error antes de pedir ayuda

Depuración de errores y detección de vulnerabilidades

- ■ 2.8 Buenas prácticas al usar IA para depuración y errores comunes

CHEATSHEET: DEPURACIÓN CON IA

✓ HACER:

- └ Incluir código completo
- └ Pegar el error exacto (traza completa)
- └ Explicar qué debería pasar
- └ Indicar lenguaje y versión
- └ Especificar qué tipo de ayuda necesitas

✗ EVITAR:

- └ "Tengo un error, ¿qué puede ser?"
- └ Enviar solo el mensaje de error sin contexto
- └ Aceptar la solución sin entenderla
- └ No probar casos borde
- └ Ignorar recomendaciones de seguridad



Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



